RescueMedAI — Complete Code & Architecture Masterbook

Line-by-Line Code Breakdown, Mathematical Formulation, and Mental Model Guide

---

Part 1: The 5-Minute Mental Model — How to Read Any Code File

To understand RescueMedAI instantly during your presentation, remember that every component revolves around **one shared data contract: IncidentState** (in src/backend/state.py).

1. **Inputs** enter through Streamlit UI (app.py -> src/frontend/main.py).
2. **Vision Module** (src/backend/vision/) inspects images, computes Sobel debris energy & flood masks, and writes hazard weights to IncidentState.road_conditions.
3. **Routing Module** (src/backend/routing/) updates edge weights on RoadNetwork graph and runs A* Search to find the safest route.
4. **Allocation Module** (src/backend/allocation/) matches trauma priority with available ALS/BLS ambulances and hospital capacities.
5. **Triage Engine** (src/backend/triage/) runs START Rules + Bayesian Network with Conservative Escalation to classify patient priority (RED/YELLOW/GREEN).
6. **Hospital Module** (src/backend/hospital/) generates SBAR telemetry text and exports printable PDF reports.

Part 2: Detailed Line-by-Line Code Walkthrough (File-by-File)

1. Incident State & Contract — src/backend/state.py

Purpose: Defines Python dataclasses that serve as the universal data contract passed across all 5 phases.

Key Classes & Fields: • DetectionItem: Stores bounding box [x1, y1, x2, y2], label, and confidence. • RoadConditionAssessment: Stores corridor damage status, debris_blockage_pct, flood_coverage_pct, and suggested_hazard_weight. • PatientRecord: Stores vitals (systolic_bp, spo2, heart_rate, respiratory_rate, consciousness). • TriageDecision: Stores final priority (RED/YELLOW/GREEN), confidence_score, and conservative_escalation_applied flag. • IncidentState: Central state holding detections, selected_route, patient, triage, and pipeline_log.

```
# Sample Code Pattern in state.py @dataclass class IncidentState: incident_id: str = field(default_factory=lambda: f"INC-{uuid.uuid4().hex[:8].upper()}") disaster_type: str = "Earthquake" detections: List[DetectionItem] = field(default_factory=list) selected_route: Optional[RouteResult] = None patient: Optional[PatientRecord] = None triage: Optional[TriageDecision] = None
```

2. Vision & Perception — src/backend/vision/detector.py

Purpose: Uses deep learning segmentation + classical image gradient analysis to detect structural rubble and floodwaters.

How the Code Works Step-by-Step: 1. **Deep Learning Model**: seg.lraspp_mobilenet_v3_large runs inference on aerial image to get class probabilities. 2. **Sobel Gradient Energy Filter (Rubble Energy)**: Computes horizontal/vertical image gradients to measure structural fracturing: gray = 0.2989*R + 0.5870*G + 0.1140*B edge = (grad_x + grad_y) / 2.0 If edge > 22.0, pixel is classified as rubble debris. 3. **Color-Space Water Mask**: Checks relative RGB channel ratios: (r > 70) & (g > 45) & (b > 15) & (r > b * 1.10) to identify mudflow/flood surface. 4. **Corridor Hazard Weights**: Calculates percentage of flood/rubble in 3 corridor bounding boxes. If flood_pct >= 20% or rubble_pct >= 28% -> Condition = Blocked/Flooded (Hazard Weight = 5.0, Impassable). 5. **System Fallback**: If PyTorch DLL is blocked by OS policy (OSError WinError 4551), HAS_TORCH = False engages seamless NumPy Sobel fallback.

```
# Sobel Gradient Energy snippet from detector.py gray = 0.2989 * r + 0.5870 * g + 0.1140 * b grad_y = np.abs(gray[1:, :] - gray[:-1, :]) grad_x = np.abs(gray[:, 1:] - gray[:, :-1]) edge = np.pad((grad_y[:, :-1] + grad_x[:-1, :]) / 2.0, ((0, 1), (0, 1)), mode='edge') rubble_mask = edge > 22.0
```

3. Road Network Graph & Routing — src/backend/routing/

Purpose: Constructs a dynamic weighted graph of city roads and computes optimal routes using A* Search.

Key Code & Mathematical Formulas: • RoadNetwork (graph.py): Represents city grid as adjacency list self.adj[u][v]. Dynamic Edge Cost Formula: get_edge_cost(u, v) = (distance_km / speed_kmh) * 60.0 * (1.0 + hazard_weight) • a_star_search() (search.py): Evaluates **f(n) = g(n) + h(n)** using heapq priority queue: - g(n): Actual accumulated travel time + vision hazard penalty from start to node *n*. - h(n): Admissible Euclidean distance heuristic from node *n* to goal hospital: h(n) = euclidean_distance(n, goal) / max_speed • compare_shortest_vs_safest(): Executes both shortest_distance_search() (ignores hazards) and a_star_search(). Proves that physical shortest route traverses hazards (high risk cost), while A* finds a safer detour saving overall travel/risk time.

```
# A* Priority Queue loop in search.py frontier = [] heapq.heappush(frontier, (h_start, 0, start)) while frontier: f, _, current = heapq.heappop(frontier) if current == goal: return reconstruct_path(came_from, current) for neighbor in graph.get_neighbors(current): tentative_g = g_score[current] + graph.get_edge_cost(current, neighbor) if tentative_g < g_score.get(neighbor, inf): came_from[neighbor] = current g_score[neighbor] = tentative_g h = graph.admissible_travel_heuristic(neighbor, goal) heapq.heappush(frontier, (tentative_g + h, tie, neighbor))
```

4. Hybrid Medical Triage Engine — src/backend/triage/

Purpose: Classifies disaster casualties into RED (Immediate), YELLOW (Urgent), or GREEN (Delayed) using dual rule-probabilistic arbitration.

Module Breakdown: 1. **Rule Engine** (rules.py): Implements START / SALT triage protocol thresholds: - SpO2 < 90% OR Respiratory Rate > 30 OR AVPU == Unresponsive -> **RED** - SpO2 < 94% OR Systolic BP < 100 mmHg -> **YELLOW** 2. **Bayesian Belief Network** (bayesian.py): Computes posterior probabilities for 5 critical pathologies (Tension Pneumothorax, Hemorrhagic Shock, TBI, Burn Shock, Stable): P(Pathology | Vitals) = P(Vitals | Pathology) * P(Pathology) / P(Vitals) 3. **Shannon Entropy (Uncertainty)**: Calculates entropy H(X) = -sum(P(x) * log2(P(x))). High entropy indicates conflicting vitals, penalizing confidence score. 4. **Conservative Escalation Matrix** (arbitrator.py): If cumulative critical probability P_critical >= 45%, patient is automatically escalated to **RED (Immediate)** regardless of rule output.

```
# Conservative Escalation in arbitrator.py rule_res = self.rule_engine.evaluate(patient) bayes_res = self.bayesian_net.infer(patient) sum_critical_prob = sum(bayes_res.hypotheses_posteriors.get(c, 0.0) for c in critical_conditions) if rule_res.suggested_priority == "RED": final_priority = "RED" elif sum_critical_prob >= 0.45: # Conservative Escalation final_priority = "RED" conservative_escalation = True explanation = "Bayesian inference detected high cumulative critical risk favoring " + bayes_res.top_condition
```

5. Allocation, Hospital Telemetry & Streamlit UI

• **Resource Allocator** (src/backend/allocation/allocator.py): Calculates score: Score = w1*TraumaMatch + w2*(1/ETA) + w3*CapacityRatio. Matches RED priority to ALS ambulances and Level 1 Trauma Centers. • **Hospital Notification & PDF** (src/backend/hospital/): Formats vitals into SBAR (Situation, Background, Assessment, Recommendation) string and builds formal pre-arrival PDF reports using ReportLab canvas. • **Frontend Orchestrator** (src/frontend/main.py): Initializes st.session_state, renders command header, scenario selector, and routes rendering into 6 interactive Streamlit tabs.

Part 3: Comprehensive Review 1 Viva Defense Cheat-Sheet

Q1: What is the main innovation of RescueMedAI compared to standard routing/triage apps?

RescueMedAI bridges the gap between vision perception, transport routing, and hospital care. Rather than treating routing or medical triage in isolation, vision hazard weights dynamically alter graph routing, while triage priority dictates ALS/BLS ambulance allocation and trauma hospital notification.

Q2: Explain the math behind A* heuristic admissibility in your routing engine.

A heuristic h(n) is admissible if it never overestimates actual travel cost to the goal. In search.py, h(n) is computed as Euclidean distance divided by maximum allowable speed (60 km/h). Since no vehicle can travel faster than max speed, h(n) is guaranteed admissible, ensuring A* produces optimal safest routes.

Q3: Why use a Naive Bayes model alongside deterministic START rules in Triage?

Rules provide fast, strict compliance with clinical guidelines. However, early shock or internal bleeding may present with borderline vitals (e.g. SpO2 91%, HR 115) that fail single rule thresholds. The Bayesian model aggregates subtle multi-vital evidence to detect cumulative risk.

Q4: How do you justify the safety of an AI model in medical triage?

First, we enforce a Conservative Escalation policy (err on the side of caution). Second, Shannon Entropy measures model uncertainty. Third, the app displays mandatory disclaimers establishing that output is educational decision support requiring human clinical review.

Q5: What are the primary dataset / benchmark standards used?

Our vision benchmark evaluation is aligned with the xBD satellite disaster dataset standards, while medical triage rule logic adheres to START (Simple Triage and Rapid Treatment) and SALT guidelines.